

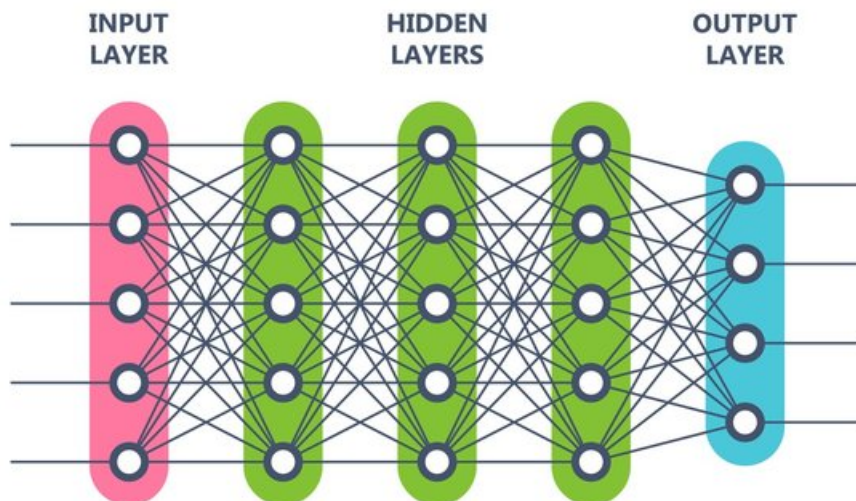


UNIVERSITÀ DI TRENTO

Final project: Learning a Value function with a Neural Network, and using it as terminal cost in an optimal control problem.

Alessandro Wegher
n.m. 249819

Date: July 2, 2025



University of Trento
Department of Industrial Engineering
Via Sommarive 14, 80123
Povo (TN), Italy

Introduction

Solving OCPs over long time horizons can be computationally very expensive and introduce quite high latency in real-time control of a robotic system. To improve the performance in the computations of the optimal trajectories, we could rely on a model that approximates the function putting in relationship the initial state for the MPC $X(t_k)$ to the cost we will have going optimally to the final state at the end of the horizon $X(t_k + N \cdot Ts)$. In order to address this challenge we will train a simple neural network as a function approximation structure that takes the initial state of the MPC x_0 as input and approximate the value of the terminal cost function $V_{nn}(x_0, N)$ also called cost-to-go function. The idea is that running the trained model is faster than evaluating the whole policy each time because the first one implies only a function evaluation compared to the iterative algorithms involved in the optimal trajectory computation with Casadi in runtime operation of the controller. The following report will present the result of this approach for two different non-linear mechanical systems : single and double pendulum. Each system has 3 main scripts with different aims:

- **Training Script:** ("SinPendTrain.py/Double_Pend_Train.py") Generates the dataset by solving many OCPs in parallel starting from random initial states configurations. After that it trains and saves a Neural Network model to be used afterward.
- **Test Script:** used to test the performances of the model
- **Comparison Script:** Compare the performances of 3 different MPC formulations (horizon M without terminal cost, horizon N using the NN without terminal cost, horizon M + N without terminal cost).

An additional script is used to define the cost computing procedure "**OCP_solve.py**" which computes the solution of the OPC in Bolza's form for both with and without terminal cost term, also referred to Mayer's term ($\Phi(x_N)$):

$$\begin{aligned} & \text{minimize} \quad \sum_{i=0}^{N-1} (w_p \cdot \|q_i\|^2 + w_v \cdot \|\dot{q}_i\|^2 + w_a \cdot \|\ddot{q}_i\|^2) + \Phi(x_N) \\ & x \in \mathbf{X}, u \in \mathbf{U} \\ & \text{subject to :} \quad x_0 = x_{init} \\ & \quad \quad \quad x_{i+1} = x_i + f(x_i, u_i) \cdot \Delta t, \quad i = 0 \dots N-1, \end{aligned}$$

The dynamic constraint is numerically implemented with a simple explicit Euler method (first order), but other numerical implementations could be used as well such as Crank-Nicholson (implicit) if we have stiff dynamics or other higher-order RK-methods, such as Dormand-Prince or RK4 even though they're much more complex to implement.

"**double_pendulum_dynamics.mw**" file where the equation of motion of the double pendulum in state-space form were found symbolically with Euler-Lagrange approach. The last python file we will use is "**neural_network.py**" the file contains the **NeuralNetwork** class used to instantiate our classifier in the training scripts.

The three analyzed cases are defined with an MPC formulation because:

- They solve a predictive OCP over a finite time horizon;
- They approximate the future behaviour of the system using the deterministic dynamics equation;
- They're intended to work recursively in real-time controllers execution.

Single Pendulum

The dynamics of this 1-DOF system are formulated using the Newton-Euler approach, implemented in the **SimPend** function. This function returns a CasADi object used for numerical integration of the system dynamics within the optimal control problem (OCP).

To generate a dataset of initial states $[q_0, \dot{q}_0]^T$ we generate a 60×60 grid with linspace.

As previously mentioned, the use of multithreading in OCP evaluation is a key factor in efficient dataset generation. It enables the construction of a larger dataset in the same wall-clock time compared to a single-threaded approach. The implementation relies on the **multiprocessing** library in Python.

For dataset generation, I adopted an OCP formulation with a horizon of $N = 200$ time steps and solved 3600 independent OCPs to construct the training set.

Neural Network Structure

- **Input Layer:** $x \in \mathbb{R}^2 \rightarrow [q_0, \dot{q}_0]^T$
- **Hidden Layers:** 3 fully connected layers, each with 64 neurons
- **Activation Function:** $\tanh(\cdot)$
- **Output Layer:** $J(x) \in \mathbb{R}$ representing the predicted terminal cost

NN's Hyperparameters

- **Learning rate :** 0.008.
- **Loss Function:** Mean Square Error.
- **Training dataset size:** 3600 data points $\rightarrow 60 \times 60$.
- **Input range:** $q_0 \in [-\pi, \pi] \quad [rad] \quad \wedge \quad \dot{q}_0 \in [-\pi, \pi] \quad [\frac{rad}{s}]$
- **batch-size:** Full-batch; all the dataset used in training, no testing batch created.
- **output scaling :** output given in terms of normalized cost over the maximum **valid** cost in the dataset **J_MAX**
- **Training loops:** 2000
- **Weights for the Lagrange running cost function:** $[w_p, w_v, w_a]^T = [1, 0.1, 0.1]^T$

The trained model is saved and later used to generate colormap plots and validate its performance in the **SinPendTest.py** script. The final loss value is acceptable ~ 0.002 . Here below some results of the Neural Network training process are shown (Figure 1). [htbp!]

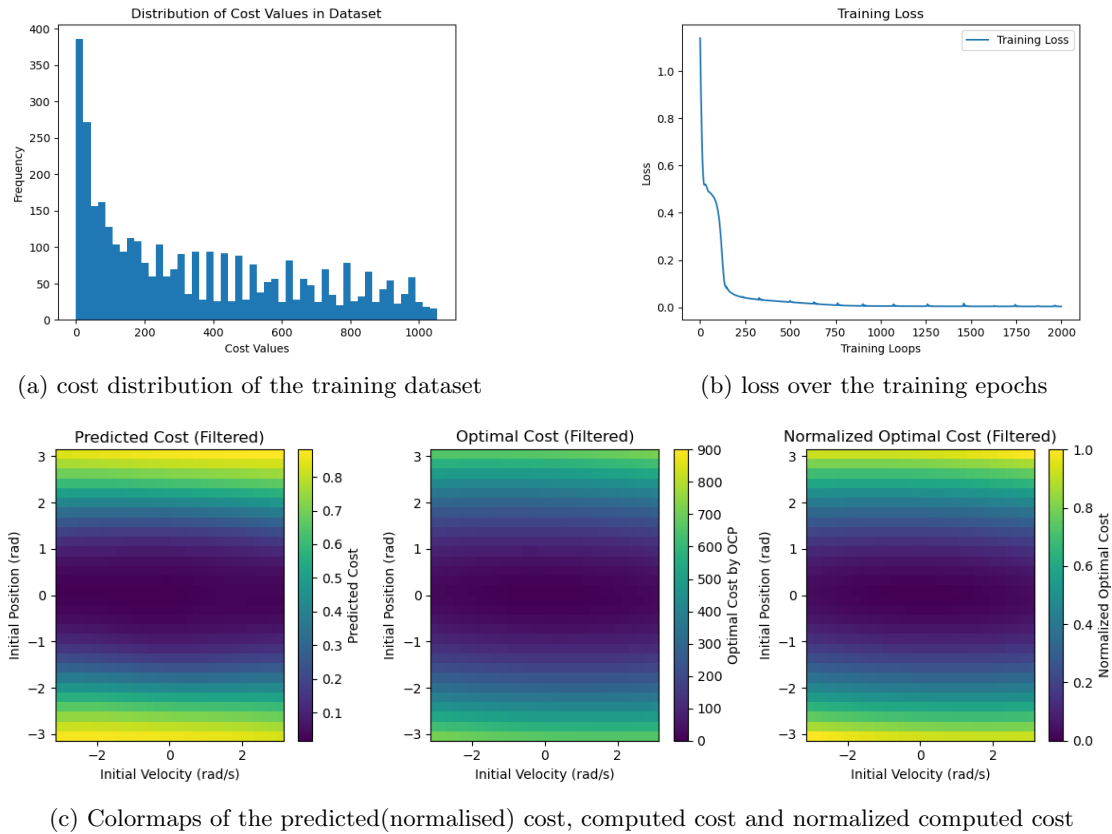


Figure 1: Neural Network Training results for the Single Pendulum

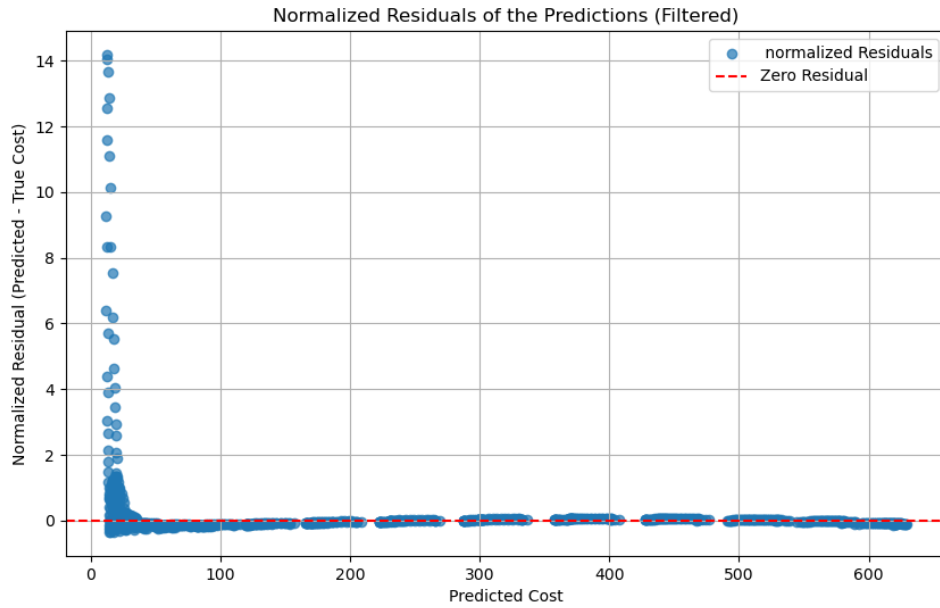


Figure 2: Normalized residuals plot for the single pendulum case

The comparison scripts "**SinPendComp**" and "**SinPendComp_multiprocess**" implement the final MPC formulations and compares the results. We calculate the differences in the cost between case 2 and 3 normalized over the computed long-time-horizon (case 3) in order to have a clear evaluation of the performances of the model which are quite good beside some outliers present during the sequence of $K = 50$ iterations of the 3 MPC implementation.

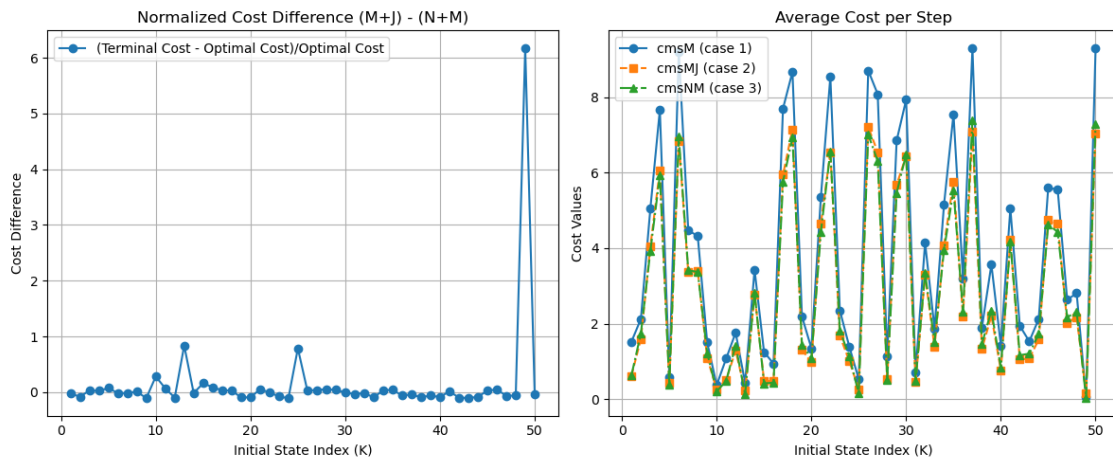


Figure 3: Comparison of the mean cost per step of the 3 MPC formulations

Some comments on the results: as expected, cases 2 and 3 yield similar cost values over the time horizon (see Figure 3, right plot), and these are consistently lower than the cost values computed over the short time horizon (case 1).

This outcome is coherent with intuition: when using a shorter horizon, the solver tends to adopt a greedier behavior due to its limited predictive capability. Consequently, control actions that appear optimal in the short term may lead to suboptimal outcomes over longer horizons.

In principle, the lowest achievable cost would correspond to the infinite-horizon case, where the control strategy is globally optimal and consistent across all iterations of the MPC controller.

Possible Improvements:

From the colorbar plot (see Figure 1.c), we observe that the neural network struggles to accurately predict the cost near the boundaries of the training domain, precisely where the cost reaches its highest values.

This difficulty arises from both the nature of the cost function being learned and the uniform sampling of the input space. In high-cost regions, the gradient of the cost function becomes steeper, meaning that for a fixed sampling density, the cost values exhibit more rapid variations. This makes the learning task harder, especially for a neural network using the tanh activation function.

Indeed, since $\tanh : \mathbb{R} \rightarrow (-1, 1)$ is bounded, it introduces saturation effects near its asymptotes. As a result, the network tends to compress large values and struggles to capture the full range of variation in the cost function, particularly in regions with steep gradients.

Switching to a different activation function such as ReLU, leaky ReLU, Swish, or ELU could mitigate this issue, as these functions either do not saturate (ReLU) or saturate less aggressively than tanh, allowing better approximation of large cost values in high-gradient regions.

Additionally, using non-uniform sampling strategies—denser near the boundaries where the cost varies more rapidly—could further improve prediction accuracy. Normalizing the cost values before training might also help mitigate the effects of large dynamic ranges. Finally, more expressive architectures such as residual or attention-based networks could be explored to better capture sharp transitions in the cost landscape.

Double Pendulum

We consider a 2-DOF nonlinear mechanical system that is fully actuated. As in the single pendulum case, the dynamics are numerically integrated using an explicit Euler scheme. However, in this case, the equations of motion are derived using the Euler-Lagrange formulation, since the actuators directly act on the generalized coordinates. This allows for a straightforward computation of the generalized forces applied to the system.

The full derivation of the dynamics is provided in the Maple file `double_pend_dynamics.mw`, where the state-space equations are also computed. To validate the dynamics implementation, an example (natural response) trajectory without input is shown in Figure 4. To

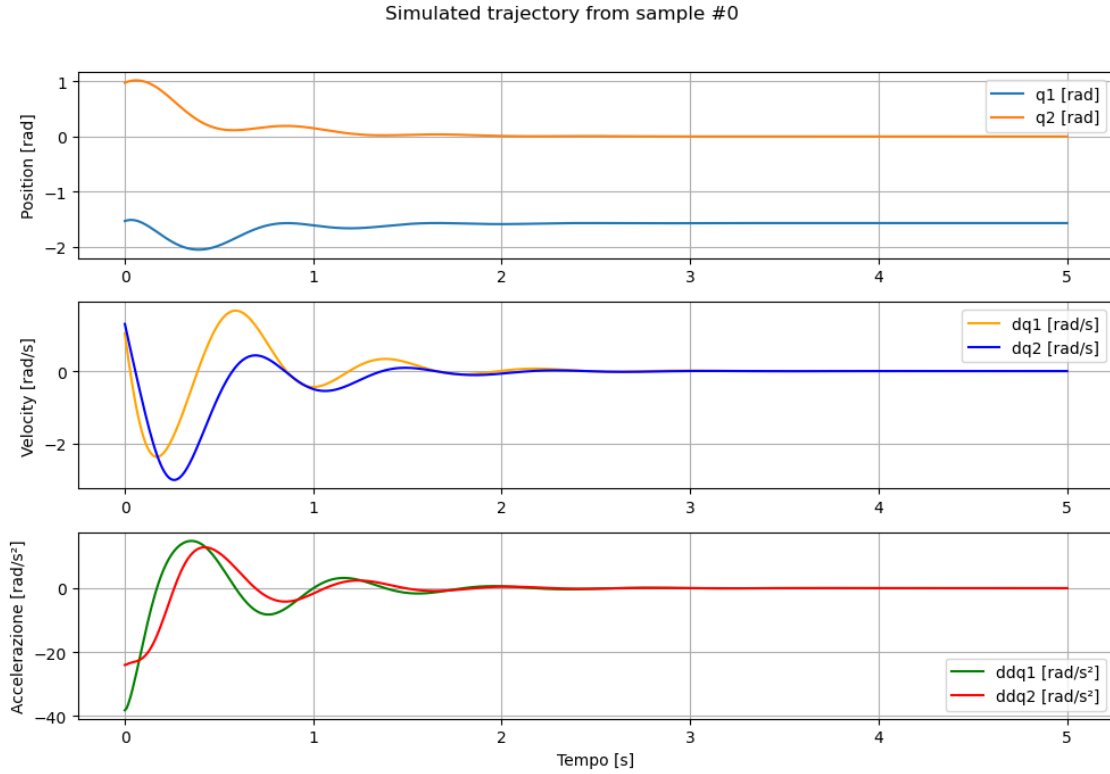


Figure 4: Example trajectory starting from a random initial state $[q_1(0), q_2(0), \dot{q}_1(0), \dot{q}_2(0)]^T$

generate the initial conditions and compute the cost values required for training the neural network, a custom `Dataset` class has been implemented. This class handles both the generation of initial states and the parallelized computation of the associated costs using multiple threads to reduce computational time. Another key component is the `Pendulum` class, which parses the parameters of the double pendulum model from its URDF file. These parameters include the link lengths, the pure rotational inertia about the actuated axes, the distances from the rotation axes to the link centers of mass, and the masses. All of these are required to compute the dynamics, including the Huygens-Steiner contributions.

In addition, a new neural network architecture has been introduced via the `ExtendedNeuralNetwork` class, which extends the capacity of the previously used `NeuralNetwork` class. The dataset used for training the neural network consists of $N_{\text{OCPs}} = 3000$ data points. The structure and hyperparameters of the neural network are detailed below.

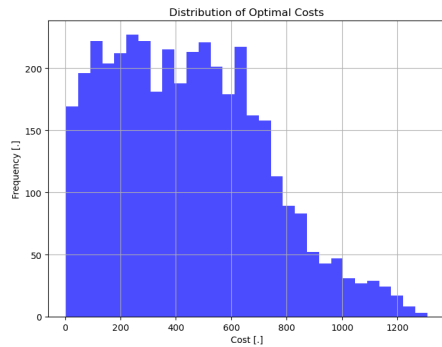
Neural Network Structure

- **Input Layer:** $x \in \mathbb{R}^4$, corresponding to the initial state $x = [q_1(0), q_2(0), \dot{q}_1(0), \dot{q}_2(0)]^T$.
- **Hidden Layers:** 6 fully connected layers, each with 128 neurons.
- **Activation Function:** $\tanh$.
- **Output Layer:** $J(x) \in \mathbb{R}$, representing the predicted terminal cost.

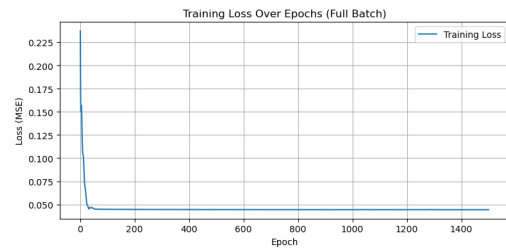
Neural Network Hyperparameters

- **Learning Rate:** 0.009
- **Loss Function:** Mean Squared Error (MSE)
- **Training Dataset Size:** 3000 data points
- **Input Range:** $\vec{q} \in \mathbb{D}$, $\dot{\vec{q}} \in \mathbb{F}$, where $\mathbb{D} = [-\pi, \pi]^2$ and $\mathbb{F} = [-2\pi, 2\pi]^2$
- **Batch Size:** Full batch
- **Output Scaling:** Normalization by the maximum cost in the dataset (J_{MAX})
- **Training Epochs:** 1500
- **Weights in the Lagrange Running Cost:** $[w_p, w_v, w_a]^T = [1, 0.05, 0.03]^T$

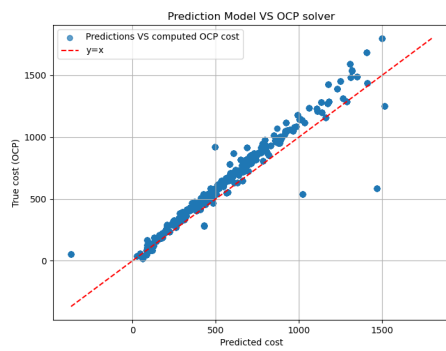
The trained model is saved and later used to generate the scatterplot and validate the performance in the **Double_Pend_Test.py** script. The final loss value is higher than the single pendulum case, it is ~ 0.04 . Here below some results of the Neural Network training process are shown (Figure 5)



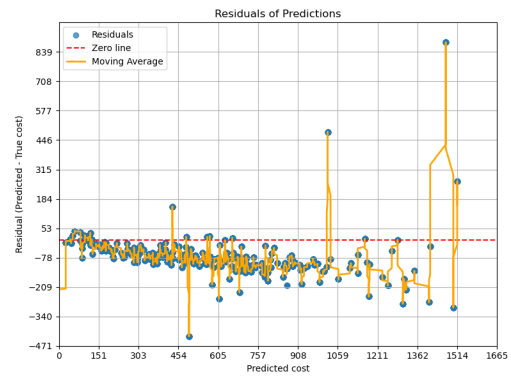
(a) cost distribution of the training dataset



(b) loss over the training epochs



(c) scatter plot predictions VS computed



(d) residuals plot

Figure 5: Neural Network Training results for the Double Pendulum

The comparison scripts `Double_Pend_comp.py` e `Double_Pend_comp_multi.py` implement the final MPC formulation for the 3 cases. The comparison are made in the same way as we've done for the single pendulum.

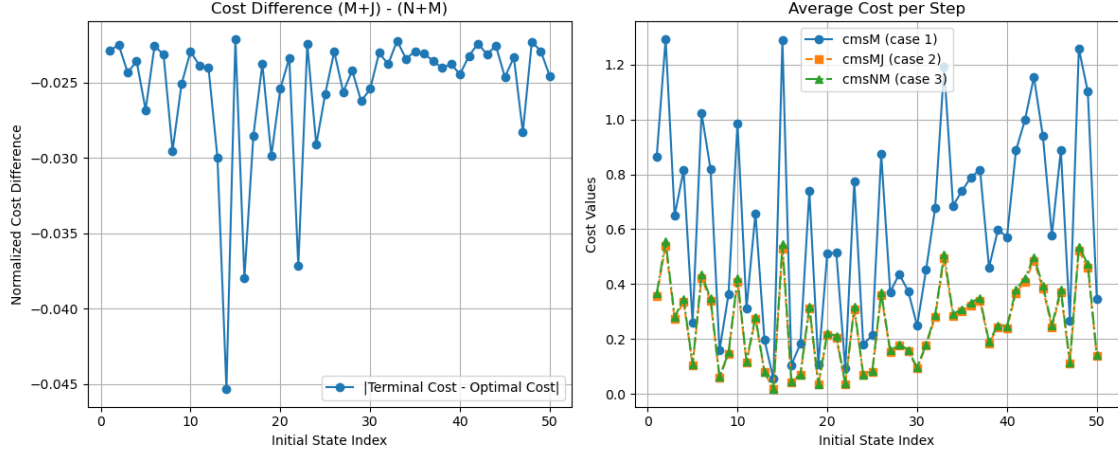


Figure 6: Comparison of the mean cost per step of the 3 MPC formulations

Discussion and Final Remarks

As already observed in the single pendulum case, the extended neural network architecture allows for accurate prediction of the terminal cost function within the training domain, enabling effective integration in an MPC framework. However, in the case of the double pendulum, the learning task is more challenging due to the increased system complexity and higher nonlinearity in the dynamics and cost function.

This is particularly evident in the residuals plot (Figure 5.d), where we can notice larger prediction errors for certain regions of the input space. These residuals reveal that while the model achieves a generally good fit, there exist specific areas, likely those with steep cost gradients or near the domain boundaries, where the neural network still struggles. This may be due to the saturation effect of the tanh activation or to under-sampling in high-variation regions.

A possible improvement could involve using alternative activation functions that are less prone to saturation, such as ReLU, Leaky ReLU or Swish, or applying non-uniform sampling strategies to better capture regions with high cost variability. Additionally, residual analysis could be leveraged to drive active learning techniques, where additional samples are collected exactly where the error is higher. Overall, the framework proves effective for cost approximation and MPC synthesis, but also highlights the importance of architecture design, sampling strategy, and post-training analysis to further improve the predictive quality of the learned models, especially in higher-dimensional nonlinear systems like the double pendulum.